思考三：显存分配与回收优化

显存管理可借鉴 Linux、JVM，以及 PostgreSQL、vLLM、Alluxio、Redis、Caffeine 的内存管理与缓存策略；重点参考冷热识别、分页复用、分层存储和回收预算，并适配 GPU 异步执行与键值缓存(Key-Value Cache，KV Cache)复用语义。本文以槽位分配、回收与淘汰为主，也统一记录服务接口等优化补丁(Patch)；候选收益需结合负载验证，缓存源码入口以当前工作区的常规分配器、Python RadixCache 路径为主。

回收语义与参考机制

- JVM 的垃圾回收(Garbage Collection，GC)主要依据可达性；被强引用缓存持有的对象，也不会因请求结束自动回收。Java 规范
- SGLang 常规 KV 池的 `free()` 归还槽位编号，底层显存池通常继续保留；槽位可复用不等于显存交还驱动。源码
- `lock_ref == 0` 表示解除请求保护；常规 RadixCache 从可淘汰叶子开始释放，仍需考虑后续复用价值。源码
- GPU 旧访问必须先于槽位的新用途完成，可通过流顺序或事件依赖保证；CPU 引用状态不足以证明安全。CUDA 文档

参考对象	原有机制	对 SGLang 的借鉴方向
Linux 内存管理	多代 LRU(Multi-Gen LRU)、工作集(Working Set)估计、主动回收	识别冷热 KV，结合容量压力与预算安排回收
JVM G1	分代回收、停顿目标	按复用情况分层，限制单次淘汰耗时
PostgreSQL 缓冲池(Buffer Pool)	固定(Pin)使用中的页，时钟扫描(Clock Sweep)选择替换页	区分安全回收条件与淘汰优先级，降低选择成本
vLLM	KV 块引用计数、空闲队列、前缀缓存	对照块管理和复用流程，比较元数据开销

1 减少槽位编号的拼接与排序

- 现状与思路：部分 `free()` 路径通过 `torch.cat` 合并编号，`need_sort` 场景还会排序；已有批量释放机制。先测编号搬运成本，若占比明显，再比较预分配编号容器，减少重复分配、复制和排序。
- 约束：保留延迟释放时的编号快照、无重复释放规则，以及确实需要排序的场景。
- 验证与入口：比较分配与回收耗时、编号复制量、GPU 内核启动次数，兼顾释放高峰；参考 TokenToKVPoolAllocator.free、批量释放与排序。

2 减少每次淘汰的候选排序开销

- 现状与思路：Python `RadixCache.evict()` 每次遍历可淘汰叶子，重新构建优先级堆；若树较大且频繁小批量淘汰，可比较增量维护淘汰顺序与当前重建堆方案。
- 约束：访问、节点拆分、锁引用变化和叶子删除都会影响顺序；维护成本可能抵消收益。
- 验证与入口：比较不同叶子数量与淘汰规模下的 CPU 耗时，以及整轮调度耗时；参考 RadixCache.evict。

3 用空闲水位与分批淘汰平滑延迟

- 现状与思路：常规分配路径在空闲容量不足时，按本次申请的缺口触发淘汰；可试验空闲低水位(Low Watermark)触发、小批补充，并为每轮维护设置时间或数量预算。
- 约束：紧急分配仍需有补足缺口或退让路径；提前淘汰可能损失即将命中的前缀。
- 验证与入口：比较淘汰耗时尖峰、词元间延迟(Inter-Token Latency，ITL)的 P99、命中率与请求退回次数；参考 evict_from_tree_cache。

4 按复用收益调整缓存保留策略

- 现状与思路：默认配置为最近最少使用(Least Recently Used，LRU)；也支持最不经常使用(Least Frequently Used，LFU)与分段 LRU(Segmented LRU，SLRU)。以现有策略为基线，结合占用字节数与重算成本评价保留价值；近期热度的计算见第 8 点。
- 约束：运行中请求的保护优先；历史热点可能过时，额外统计也有成本。
- 验证与入口：在重复前缀、一次性长请求和热点切换负载下，比较命中率、重算量与端到端延迟；参考 淘汰策略与 SLRU。

5 让淘汰策略感知即将调度的请求

- 现状与思路：调度器已有最长前缀匹配(Longest Prefix Match，LPM)和等待队列内的前缀复用逻辑；可探索复用现有匹配结果，为即将使用的缓存增加短期保留权重，减少刚淘汰就重算的情况。

- 约束：保留权重需有上限和失效条件，避免等待请求占满保护容量，或低命中请求长期排队。
- 验证与入口：比较淘汰后短期重算量、首词元延迟(Time to First Token，TTFT)、队列等待时间与公平性；参考 SchedulePolicy.calc_priority、RadixCache.evict。

6 降低分页造成的页内空闲

- 现状与思路：分页分配存在末页空位；例如独占页面时，18 个词元(Token)按每页 16 个分配，需 32 个槽位。可在注意力后端支持的页大小内，按请求长度和并发量比较页内碎片(Internal Fragmentation)。
- 约束：小页会增加管理开销，也可能改变算子效率；应与缺少空闲页、前缀保留占用分别统计。
- 验证与入口：比较实际有效槽位占比、可服务并发量、吞吐量与 ITL，避免只追求空间利用率；参考 PagedTokenToKVPoolAllocator。

7 按成本选择分层缓存与逐层预取

- 现状与已有机制：HiRadixCache 已有备份、回载和存储预取；分层缓存(Hierarchical Cache，HiCache)支持传输与计算重叠(Compute-Transfer Overlap)：计算第 N 层时，回载后续层的 KV，读取前按层等待完成事件。
- 适用范围：提前搬运的是主机内存中已有的历史 KV；当前新 token 的下一层 KV 仍依赖上一层输出，不能提前凭空生成。
- 思路：比较保留显存、卸载(Offload)到主机、丢弃后重算的成本，再按可用容量与带宽调整预取(Prefetching)提前量和预算。
- 容量条件：有可用 KV 槽位才可接收回载数据；显存空闲不直接代表收益，还需考虑传输带宽、主机容量、排队和复用概率。
- 边界：KV 已在 GPU 显存时，进一步预取属于算子内部访存优化；访问或传输中的槽位不得提前覆盖，避免预取挤占运行请求。
- 验证：分层等待时间、搬运字节数、无效预取比例、重算耗时，以及 TTFT 和 ITL；检查搬运是否与计算实际重叠。
- 入口：start_loading、逐层读取等待。

8 用近期访问频率与时间衰减计算淘汰热度

- 局限：LRU 只比较最后访问时间，可能先淘汰频繁复用但暂时安静的 A，保留刚被访问一次的 B。
- 参考一：LRFU 论文有轨迹评测，Alluxio已有实现；旧分数按访问间隔指数衰减，命中加 1，经典实现使用逻辑访问时钟。
- 参考二：Caffeine W-TinyLFU有公开轨迹评测；按累计访问量触发计数减半，用于缓存准入(Admission)，判断新条目是否值得挤掉旧条目。
- 参考三：Redis LFU使用随时间衰减的近似频次计数，是正式支持的产品机制；具体计数和衰减规则与下述 EWMA 不同。
- 候选公式：固定时间窗 Δ 使用指数加权移动平均(EWMA)，`H_t = α × N_t + (1 − α) × H_(t−1)`；N 为该窗实际复用次数、H 为热度、`0 < α < 1`。
- 更新：无访问的窗也按 `N=0` 衰减，可按空窗数惰性计算；α 越大越偏重近期，Δ 决定时间尺度；现有 LFU、SLRU 的 `hit_count` 未按时间衰减。
- 保护与口径：排除 `lock_ref > 0`，只在可淘汰叶子中比较热度；实际复用按请求去重，节点拆分需继承历史，避免探测和分块重复计数。
- 验证：通用缓存的评测不等于 SGLang 收益；对比 LRU、LFU、SLRU，测热点切换时的命中率、重算量与策略开销。

9 按需压实统一显存池以改善容量共享

- 先定位：显存未占满可能来自运行预留、页尾空位或子池容量失衡；空闲显存不一定全属于 KV 分配预算。
- 常规分页：`free_pages=[2,5,9]` 可直接满足 3 页申请，页间不要求连续；分配器通常无需搬动存活 KV 来合并这些空页。
- 适用场景与已有机制：多个子池共享一块缓冲区时，压实(Compaction)可把一侧子池内部的洞变成边界空闲区，让另一侧扩容；当前工作区的 UnifiedKVPool已有这种布局，但 enable_unified_memory默认关闭，不能视为常规分页池的默认行为。
- 搬移过程：MultiEndedAllocator._flush用存活页填洞、收缩占用边界，再更新虚实映射；上层虚拟编号保持不变。
- 边界与成本：压实不增加空闲总字节，整页搬移不消除页尾空位；复制与同步消耗带宽和时间；`merge_and_sort_free()` 仅排序编号，不搬 KV。
- 验证：比较整理前后的可分配容量、子池空洞、搬运字节数和 ITL；优先验证已有延迟压实机制，避免每次释放都搬运。

10 借鉴 Java 线程池的任务调度

借鉴 Java 线程池(ThreadPoolExecutor)的队列管理、并发控制和拒绝策略，结合 token 预算与 GPU 显存容量优化 SGLang 的请求调度，相关流转与回收过程见思考一：SGLang 请求任务流转与 KV Cache 回收。

验证顺序与淘汰策略对照测试

- 先测第 1、2 点的路径开销，再决定是否改数据结构；随后验证第 3 点的延迟收益。
- 正确性：复用已接入 CPU 持续集成(CI)的 策略单测和 SLRU 用例；新策略需覆盖衰减排序、锁保护和节点拆分。
- 工具：端到端压测使用 `python -m sglang.benchmark.serving`；缓存微基准可参考，主要面向 UnifiedRadixCache。
- 区分：齐普夫分布(Zipf)生成冷热前缀请求，轨迹回放(Trace Replay)按时间戳发请求；它们是测试负载，不是已有的衰减淘汰策略。
- 场景：覆盖稳定热点、热点短暂沉默、热点切换和一次性请求冲击；静态 Zipf 不自动模拟热点切换，需另外构造时序；工作集应超过缓存容量并确认触发淘汰。
- 对照：比较 LRU、LFU、SLRU 与新策略；固定模型、后端、KV 容量、调度配置、请求序列及到达时间、并发和预热方式，每轮仅改变策略。
- 指标：服务压测的 `--cache-report` 统计 token 命中率，配合吞吐、TTFT/ITL 的 P99、重算量及淘汰耗时；可用 `--output-file` 保存 JSONL。
- 判定：用多组随机种子重复成对对照，检查收益及尾延迟退化；同时记录请求退回量，区分池内利用率与驱动显存占用，正确性通过不等于性能更好。

patch-1 优化错误码(Error Code) 待实现

- 场景：普通非预填充／解码分离(Prefill-Decode Disaggregation，PD)模式设置 `--max-queued-requests 100` 后，队列满触发拒绝；默认 `None` 不设固定等待数量上限，PD 模式不使用此限制。配置
- 问题：队列满、被高优先级请求替换、等待超时，以及禁用优先级时拒绝携带优先级的请求，均返回 `503`，主要靠 `message` 区分；OpenAI 的整数 `code` 也直接取 HTTP 状态码，客户端难以据此区分原因与重试方式。调度器、响应构造、ErrorResponse
- 思路：区分 HTTP 状态码与稳定、可机读的业务错误标识，候选如 `queue_full`、`priority_preempted`、`waiting_timeout`；具体字段及状态码映射在兼容性评审后确定。临时过载与参数／配置不兼容分别表达，正常达到 `--max-running-requests` 后的排队不应当作错误。
- 验证与入口：覆盖以上拒绝原因，检查流式与非流式传递及客户端兼容性；沿 `Scheduler._abort_on_queued_limit()` → TokenizerManager._handle_abort_finish_reason → OpenAI 错误响应核对。

术语与生词

缩写或术语	英文全称	中文释义	简明英文释义
JVM	Java Virtual Machine	Java 虚拟机	Runtime that executes Java bytecode.
G1	Garbage-First	一种分区域的垃圾收集器	A region-based garbage collector.
LRFU	Least Recently/ Frequently Used	综合近期性与频率的淘汰策略	Combining recency and frequency through decayed reference scores.
W-TinyLFU	Window TinyLFU	带窗口的 TinyLFU 缓存策略	Combining a recency window with frequency-based admission.
EWMA	Exponentially Weighted Moving Average	指数加权移动平均	An average that gives exponentially less weight to older observations.
P99	99th Percentile	第 99 百分位数	A value at or below which 99 percent of observations fall.